

Blockchain Assignments – Decentralized Applications

Assignment 1: Smart Contract Development & Testnet Deployment

Objective

To understand how to develop, compile, and deploy a smart contract on an Ethereum testnet.

Tools Used

- **Remix IDE (Online Solidity IDE)**
 - **MetaMask Wallet**
 - **Sepolia Testnet**
 - **Sepolia Faucet**
 - **Sepolia Etherscan**
-

Step 1: Setup Environment

1. Install MetaMask

1. Go to Chrome Extensions.
 2. Search for MetaMask.
 3. Install and create a wallet.
 4. Save the Secret Recovery Phrase securely.
-

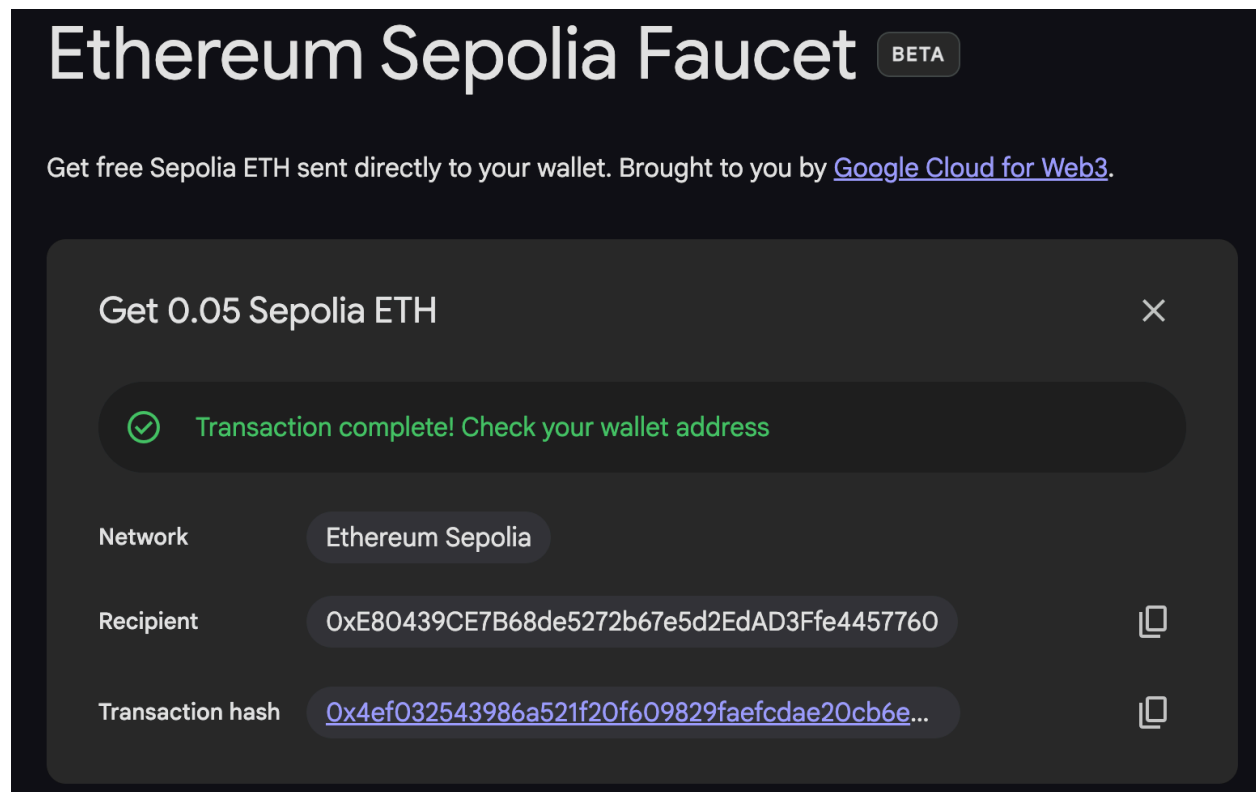
Blockchain Assignments – Decentralized Applications

2. Add Sepolia Test Network

1. Open MetaMask.
2. Click Network dropdown.
3. Select **Sepolia Test Network**.

3. Get Test ETH from Faucet

1. Copy your wallet address.
2. Go to Sepolia Faucet.
3. Paste address and request ETH.

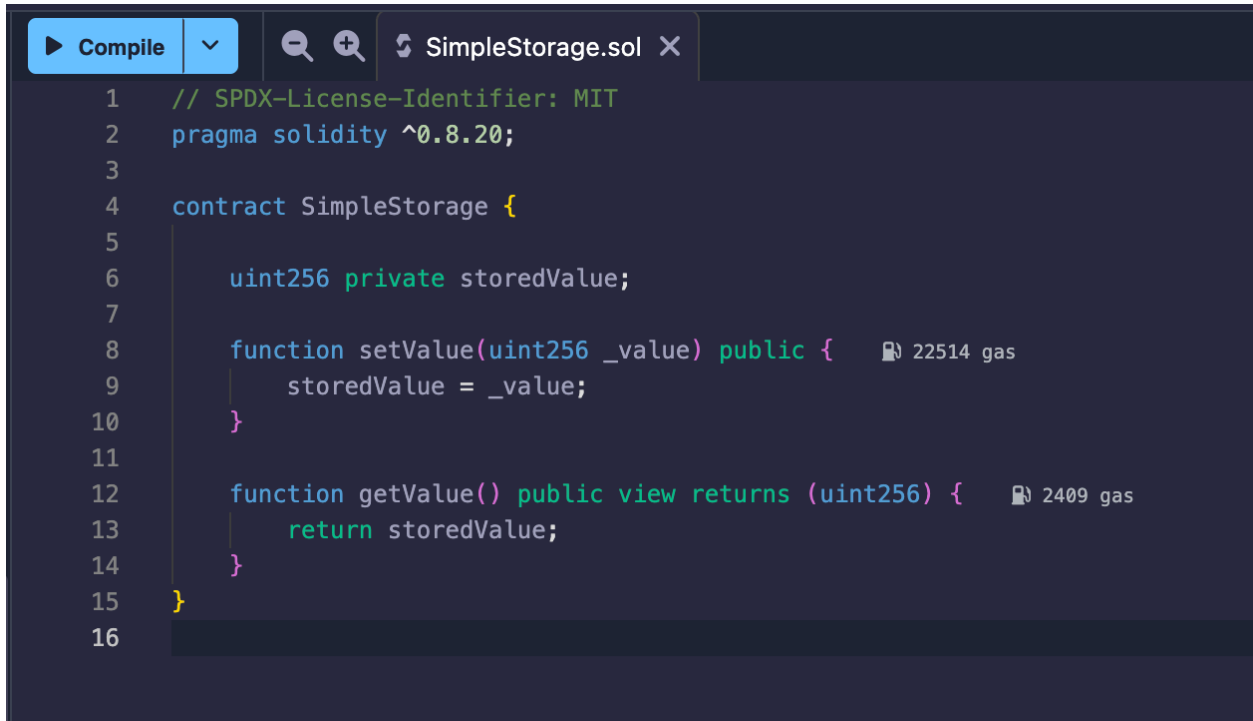


Step 2: Write Smart Contract

Blockchain Assignments – Decentralized Applications

Open Remix IDE.

Create file: `SimpleStorage.sol`

A screenshot of the Remix IDE interface. The top bar shows a 'Compile' button, a dropdown menu, and a file tab for 'SimpleStorage.sol'. The main editor area displays Solidity code for a 'SimpleStorage' contract. The code includes a license comment, a pragma statement for Solidity version 0.8.20, and two functions: 'setValue' and 'getValue'. The 'setValue' function is marked as 'public' and has a gas cost of 22514. The 'getValue' function is marked as 'public view returns' and has a gas cost of 2409. The code is syntax-highlighted with colors: blue for keywords, green for comments, and various colors for identifiers and literals. Line numbers 1 through 16 are visible on the left side of the editor.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract SimpleStorage {
5
6     uint256 private storedValue;
7
8     function setValue(uint256 _value) public { 22514 gas
9         storedValue = _value;
10    }
11
12    function getValue() public view returns (uint256) { 2409 gas
13        return storedValue;
14    }
15 }
16
```

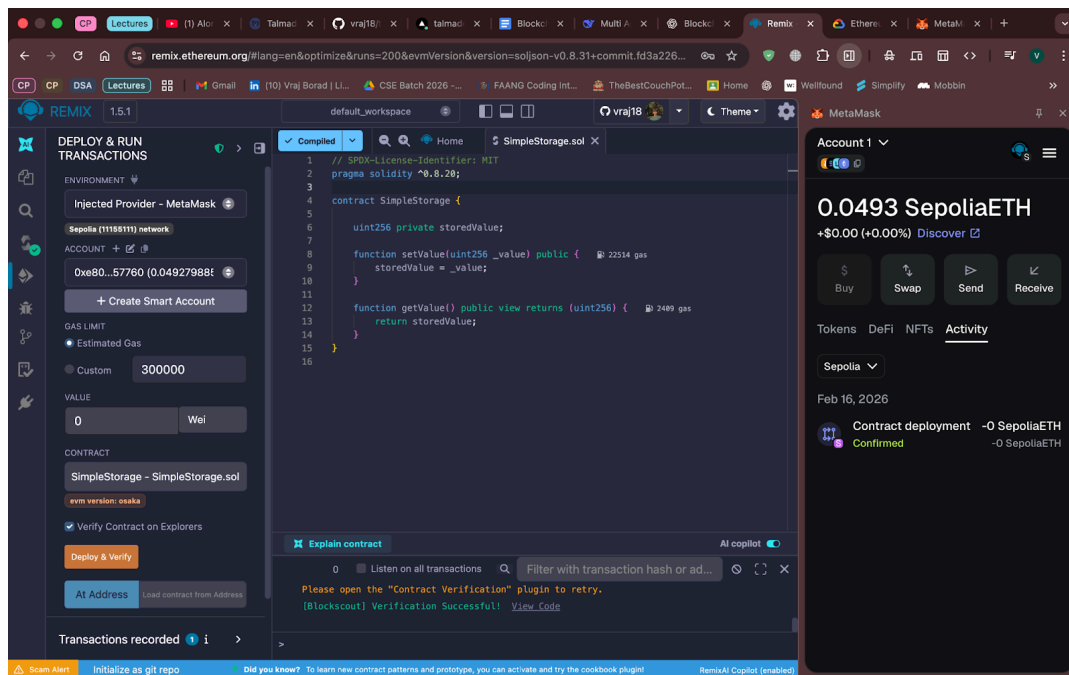
Step 3: Compile Contract

1. Go to Solidity Compiler tab.
2. Select version 0.8.x.
3. Click Compile.

Step 4: Deploy Contract

Blockchain Assignments – Decentralized Applications

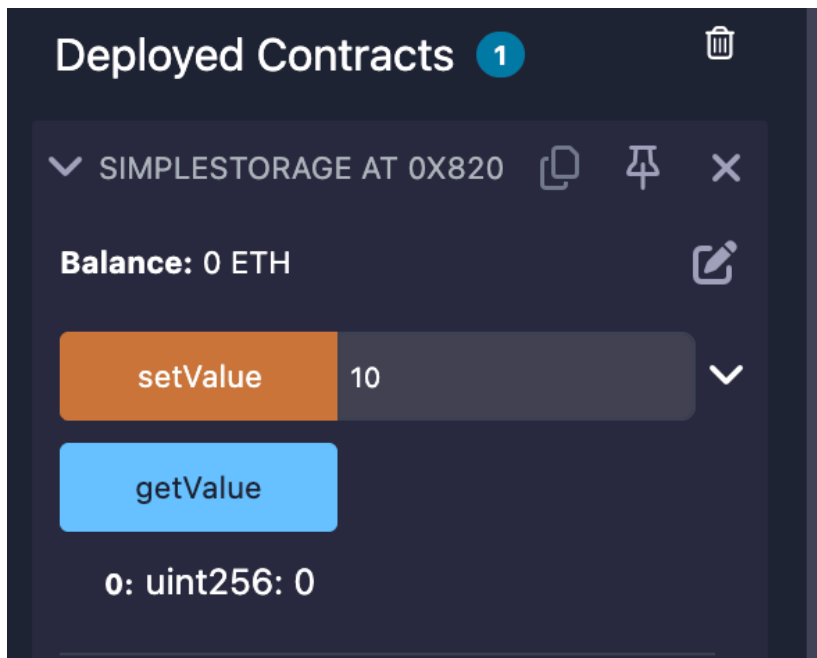
1. Go to Deploy & Run Transactions tab.
2. Environment → Injected Provider (MetaMask).
3. Select SimpleStorage.
4. Click Deploy.
5. Confirm in MetaMask.



Step 5: Test Contract

1. Enter a number in `set()`.
2. Click transact.
3. Call `get()` to retrieve value.

Blockchain Assignments – Decentralized Applications



Assignment 2: ERC-20 Token Creation

Objective

To create and deploy a custom ERC-20 token.

Step 1: Understanding ERC-20

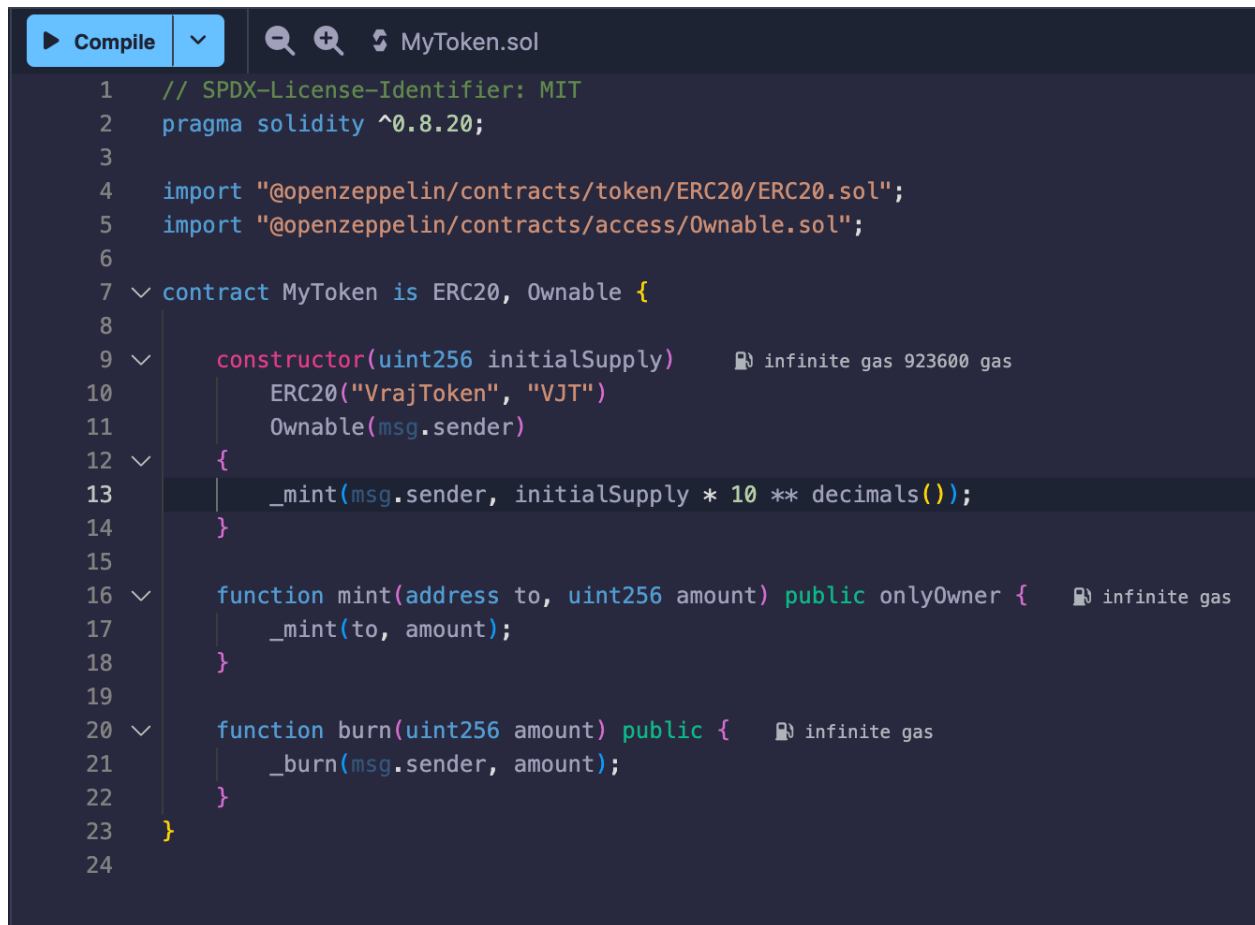
ERC-20 is a token standard defining:

- totalSupply
- balanceOf
- transfer
- approve
- transferFrom

Blockchain Assignments – Decentralized Applications

Step 2: ERC-20 Token Contract

Create `MyToken.sol`

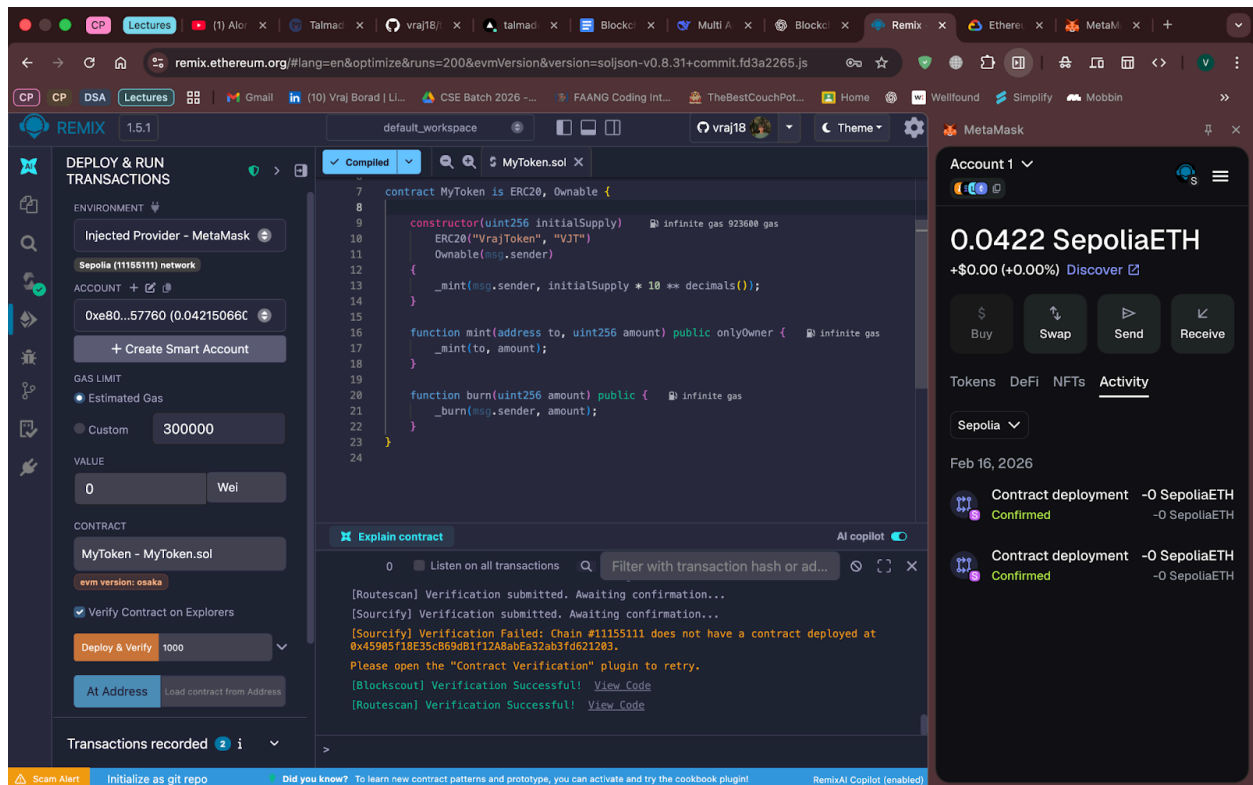
A screenshot of a Solidity code editor interface. At the top, there is a toolbar with a 'Compile' button, a dropdown menu, and icons for search, zoom, and refresh. The file name 'MyToken.sol' is displayed. The code is written in Solidity and includes comments for SPDX license and pragma solidity version. It imports ERC20 and Ownable contracts. The main contract 'MyToken' inherits from ERC20 and Ownable. It has a constructor that initializes the token name as 'VrajToken' and symbol as 'VJT', and mints the initial supply. It also has functions for minting and burning tokens.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5 import "@openzeppelin/contracts/access/Ownable.sol";
6
7 contract MyToken is ERC20, Ownable {
8
9     constructor(uint256 initialSupply) {
10         ERC20("VrajToken", "VJT")
11         Ownable(msg.sender)
12     }
13     _mint(msg.sender, initialSupply * 10 ** decimals());
14 }
15
16 function mint(address to, uint256 amount) public onlyOwner {
17     _mint(to, amount);
18 }
19
20 function burn(uint256 amount) public {
21     _burn(msg.sender, amount);
22 }
23 }
24
```

Blockchain Assignments – Decentralized Applications

Deployment

Same steps as Assignment 1.



Blockchain Assignments – Decentralized Applications

Testing

- Transfer tokens
- Check balances
- Mint tokens
- Burn tokens

The screenshot displays the Remix IDE interface with a Solidity contract named `MyToken.sol` open in the editor. The contract is an ERC20 token with a total supply of 10,000 tokens (10 * 10^4). It includes functions for minting and burning tokens. The left sidebar shows the 'DEPLOY & RUN TRANSACTIONS' panel with a list of deployed contracts, including 'MYTOKEN AT 0x10E...9254'. The right sidebar shows a MetaMask wallet connected to the Sepolia network, displaying a balance of 0.00373 SepoliaETH. The bottom status bar indicates that the Remix AI Copilot is enabled.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5 import "@openzeppelin/contracts/access/Ownable.sol";
6
7 contract MyToken is ERC20, Ownable {
8
9     constructor(uint256 initialSupply) ERC20("VrajToken", "VJT") Ownable(msg.sender) {
10         _mint(msg.sender, initialSupply * 10 ** decimals());
11     }
12
13     function mint(address to, uint256 amount) public onlyOwner {
14         _mint(to, amount);
15     }
16
17     function burn(uint256 amount) public {
18         _burn(msg.sender, amount);
19     }
20 }
```

Account 1
0.00373 SepoliaETH
+\$0.00 (+0.00%) Discover

Tokens DeFi NFTs Activity

Sepolia

Feb 16, 2026

- Sent Token -0 SepoliaETH Confirmed
- Mint -0 SepoliaETH Confirmed
- Burn -0 SepoliaETH Confirmed
- Contract deployment -0 SepoliaETH Confirmed
- Contract deployment -0 SepoliaETH Confirmed

Scam Alert Initialize as git repo Did you know? To learn new contract patterns and prototype, you can activate and try the cookbook plugin! RemixAI Copilot (enabled)

Blockchain Assignments – Decentralized Applications

Assignment 3: Decentralized Voting System

Objective

Build secure voting smart contract.



```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract Voting {
5
6     struct Candidate {
7         string name;
8         uint voteCount;
9     }
10
11     address public owner;
12
13     mapping(address => bool) public registeredVoters;
14     mapping(address => bool) public hasVoted;
15
16     Candidate[] public candidates;
17
18     constructor(string[] memory candidateNames) {
19         owner = msg.sender;
20
21         for(uint i = 0; i < candidateNames.length; i++) {
22             candidates.push(Candidate(candidateNames[i], 0));
23         }
24     }
25
26     modifier onlyOwner() {
27         require(msg.sender == owner, "Not owner");
28         _;
29     }
30
31     function registerVoter(address voter) public onlyOwner {
32         registeredVoters[voter] = true;
33     }
34
35     function vote(uint candidateIndex) public {
36         require(registeredVoters[msg.sender], "Not registered");
37         require(!hasVoted[msg.sender], "Already voted");
38
39         hasVoted[msg.sender] = true;
40         candidates[candidateIndex].voteCount++;
41     }
42
43     function getVotes(uint index) public view returns(uint) {
44         return candidates[index].voteCount;
45     }
46 }
47
```

Blockchain Assignments – Decentralized Applications

Security Measures

- One vote per address
- Public transparency
- Immutable results

The screenshot displays the Remix IDE interface for developing and deploying a Solidity contract. The central editor shows a Solidity contract named `Voting` with the following code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract Voting {
5
6     struct Candidate {
7         string name;
8         uint voteCount;
9     }
10
11     address public owner;
12
13     mapping(address => bool) public registeredVoters;
14     mapping(address => bool) public hasVoted;
15
16     Candidate[] public candidates;
17
18     constructor(string[] memory candidateNames) {
19         owner = msg.sender;
20
21         for(uint i = 0; i < candidateNames.length; i++) {
22             candidates.push(Candidate(candidateNames[i], 0));
23         }
24     }
25 }
```

The left sidebar shows the **DEPLOY & RUN TRANSACTIONS** panel. It displays the contract's balance (0 ETH) and provides a `registerVoter` button. Below this, the **VOTE** section allows setting a `candidateIndex` and a `hasVoted` address. The **CALLDATA** section shows the `owner` and `registeredVot...` parameters. A **Transact** button is available for executing the transaction.

The bottom panel shows the **EXPLAIN CONTRACT** section, which displays a list of transactions. The first transaction is a `Contract deployment` with a gas cost of 0 SepoliaETH. The second transaction is a `Contract interaction` with a gas cost of 0 SepoliaETH. The third transaction is a `Contract deployment` with a gas cost of 0 SepoliaETH. The fourth transaction is a `Contract interaction` with a gas cost of 0 SepoliaETH. The fifth transaction is a `Contract deployment` with a gas cost of 0 SepoliaETH.

The right sidebar shows the **MetaMask** interface, displaying the account balance (0.0333 SepoliaETH) and a list of recent transactions, including `Contract interaction` and `Contract deployment` transactions.

Blockchain Assignments – Decentralized Applications

candidates	uint256	▼
getVotes	0	▼
o: uint256: 1		
hasVoted	address	▼
owner		

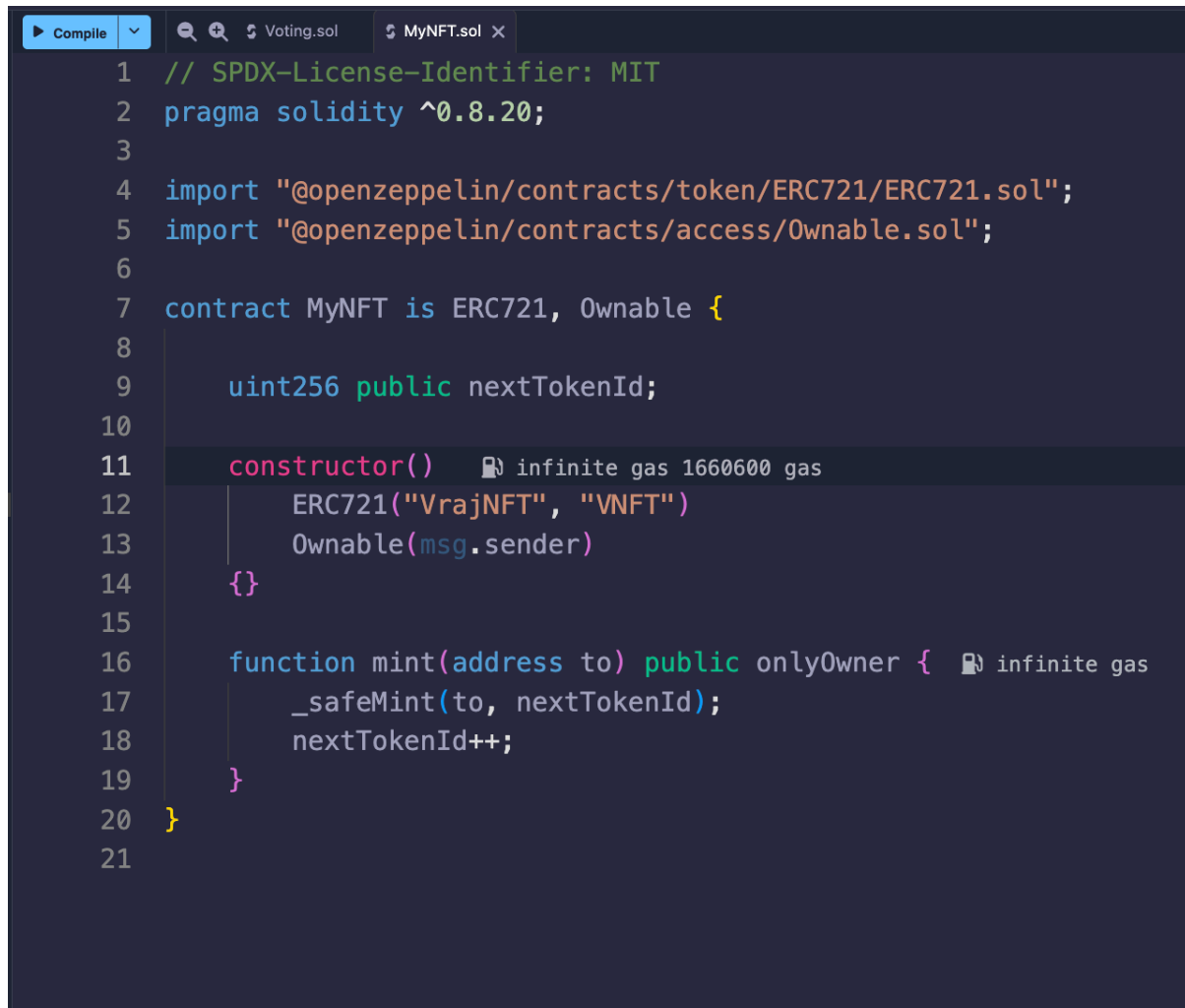
```
data: 0x012...00000 logs: 0 hash: 0xef7...1e899  
transact to Voting.vote pending ...  
  
transact to Voting.vote errored: Error occurred: execution reverted: "Already voted" (action="estim  
execution reverted: "Already voted" (action="estimateGas", data="0x08c379a000000000000000000000000000000000000000000000000000000000")  
  
If the transaction failed for not having enough gas, try increasing the gas limit gently.
```

Blockchain Assignments – Decentralized Applications

Assignment 4: NFT Marketplace

ERC-721 Contract

Blockchain Assignments – Decentralized Applications



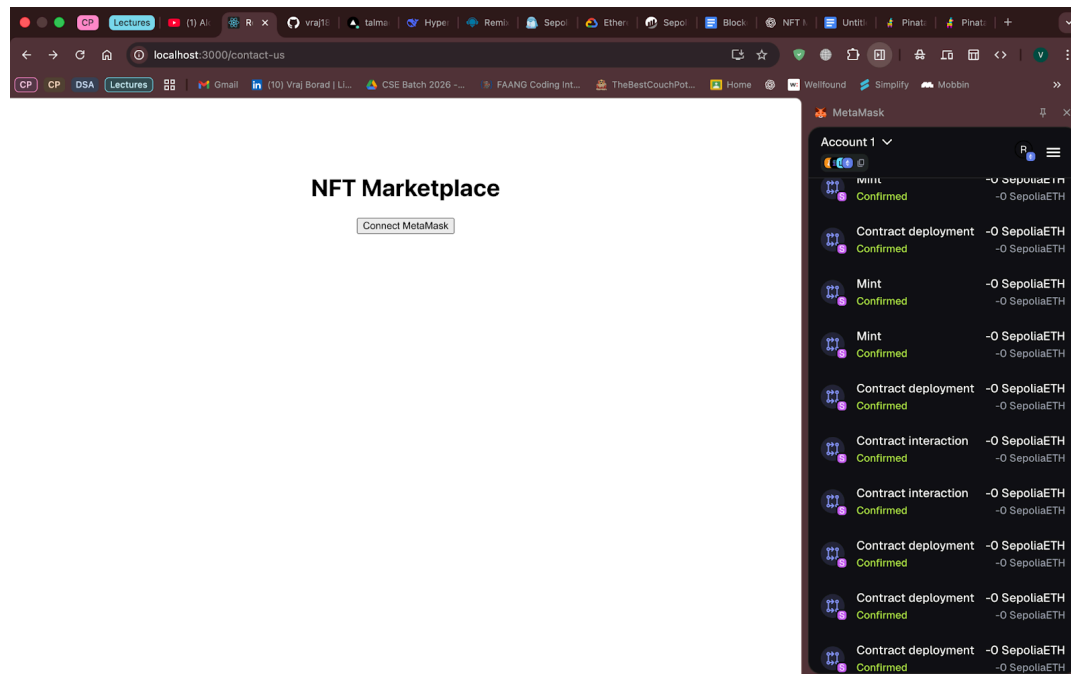
```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
5 import "@openzeppelin/contracts/access/Ownable.sol";
6
7 contract MyNFT is ERC721, Ownable {
8
9     uint256 public nextTokenId;
10
11     constructor() infinite gas 1660600 gas
12     ERC721("VrajNFT", "VNFT")
13     Ownable(msg.sender)
14     {}
15
16     function mint(address to) public onlyOwner { infinite gas
17         _safeMint(to, nextTokenId);
18         nextTokenId++;
19     }
20 }
21
```

Web3 Frontend

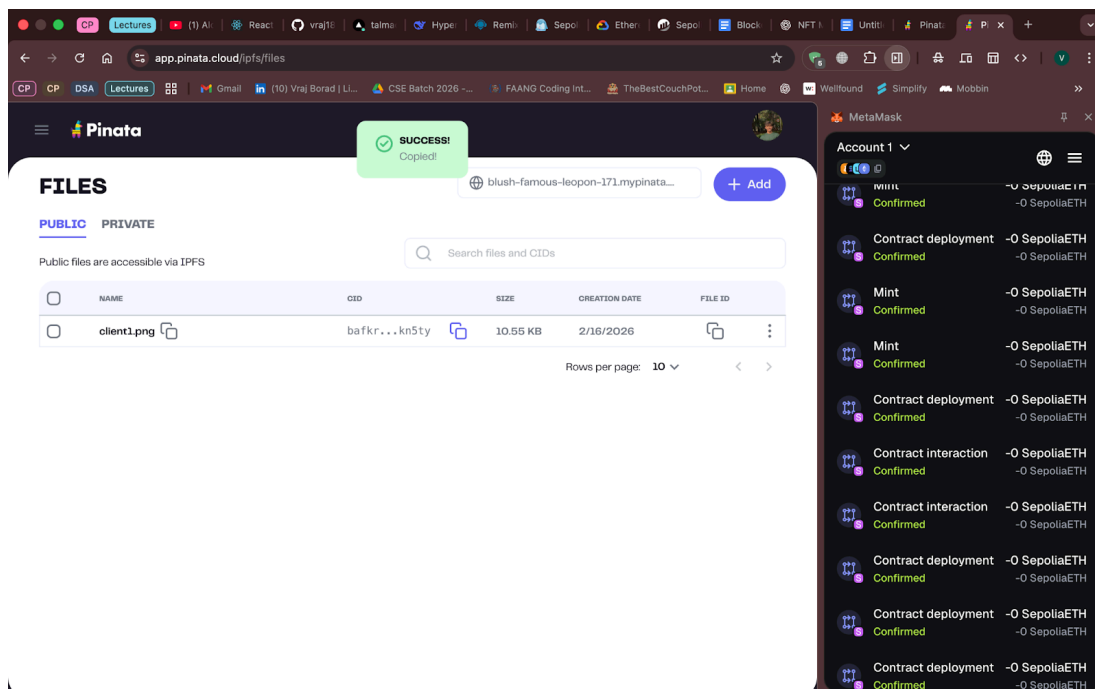
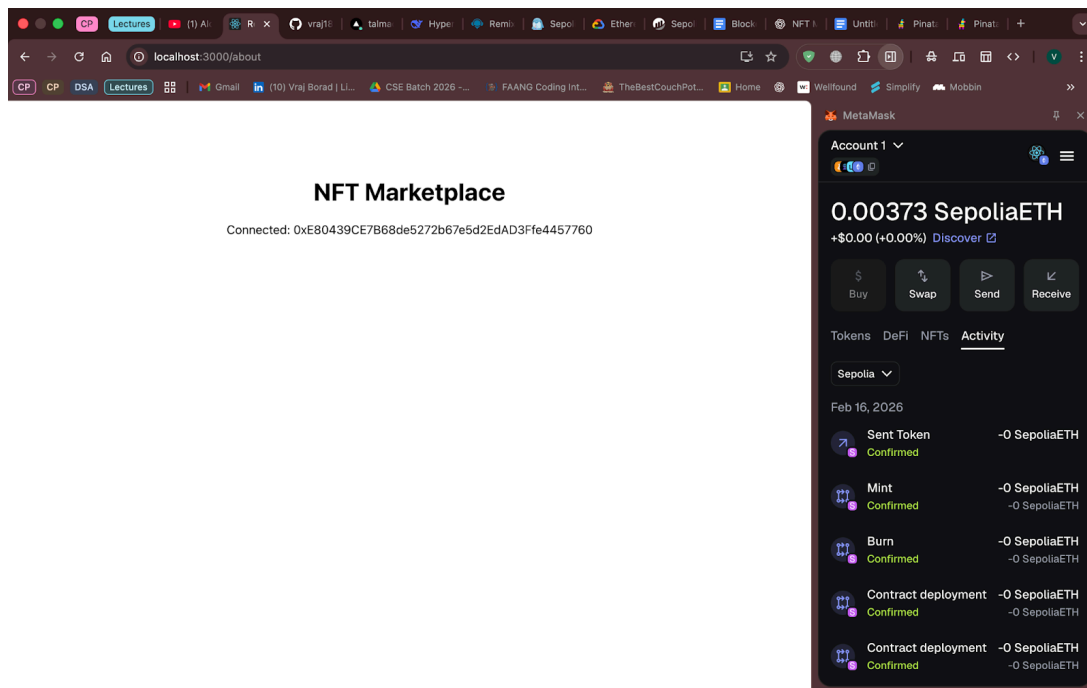
- React App
- MetaMask connection
- Display token ID

Blockchain Assignments – Decentralized Applications

block number	10270320
contract address	0x41ec87e5F84F7e65c0a0E11a5218f53a467f9877
from	0xE80439CE7B68de5272b67e5d2EdAD3Ffe4457760
to	MyNFT.(constructor)
transaction cost	1930159 gas
decoded input	{}
decoded output	-
logs	[{ "from": "0x41ec87e5F84F7e65c0a0E11a5218f53a467f9877", "topic": "0x8be0079c531659141344cd1fd0a4f28419497f9722a3daafe3b4186f6b6457e0" }, { "event": "OwnershipTransferred",



Blockchain Assignments – Decentralized Applications



Assignment 5: Hyperledger Fabric

Blockchain Assignments – Decentralized Applications

Tools

- Docker
 - Docker Compose
 - Hyperledger Fabric
-

Steps

1. Install Docker.
2. Download Fabric Samples.

Run test network:

`./network.sh up`

- 3.
4. Deploy chaincode.
5. Test invoke/query.



[Insert Screenshot: Fabric Network Running]



[Insert Screenshot: Chaincode Execution]

Challenges Faced

- Docker errors
 - Chaincode deployment issues
 - CLI configuration confusion
-